Introduction to Unsupervised Learning
clustering

Dr. Zakarya Farou

Data Science and Engineering Department

zakaryafarou@inf.elte.hu

Plan

1. Introduction
2. Clustering
3. Examples of clustering algorithms
4. Open problems in clustering
5. Bibliography

Outline

1 Introduction

2 Clustering

3 Examples of clustering algorithms

4 Open problems in clustering

5 Bibliography

What is unsupervised learning ?

Unsupervised learning *is a Machine Learning task the aim of which is to find hidden structures and patterns in* unlabeled data.
There are several tasks linked to unsupervised learning:

- Data partitioning (or clustering)
- Anomalies detection
- Unsupervised neural-networks
- Latent variables model learning

What is unsupervised learning ?

Unsupervised learning *is a Machine Learning task the aim of which is to find hidden structures and patterns in* unlabeled data.
There are several tasks linked to unsupervised learning:

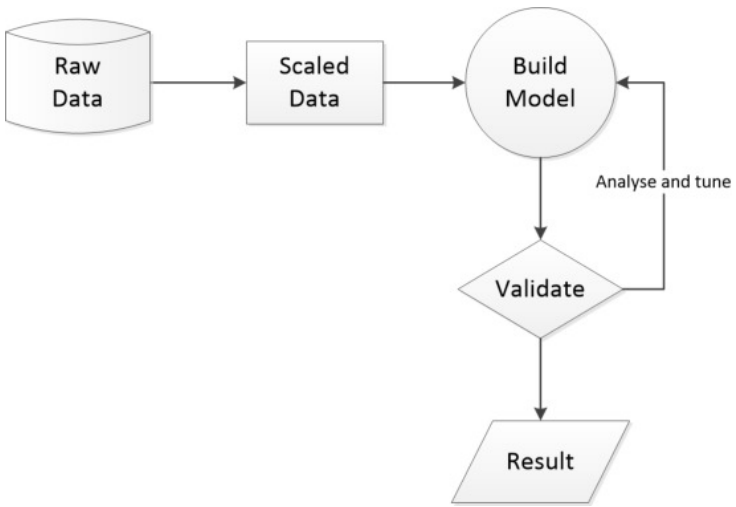
- Data partitioning (or clustering)
- Anomalies detection
- Unsupervised neural-networks
- Latent variables model learning

Remark

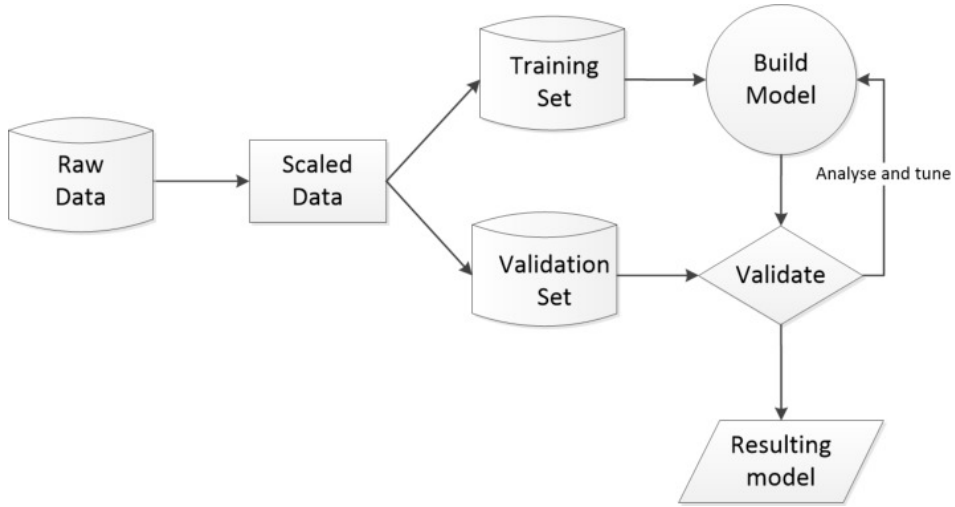
Unlike statistical analysis methods , unsupervised learning focuses on links and similarities between the data themselves.

What is unsupervised learning ?

Unsupervised learning is said to be "*unsupervised*" because it finds structures and build a model from completely unlabeled data. It therefore differs from **supervised learning** (Cf. Lecture 5) which builds a model given already labeled data.



(a) Unsupervised learning flow



(b) Supervised learning flow

Data partitioning : Definition

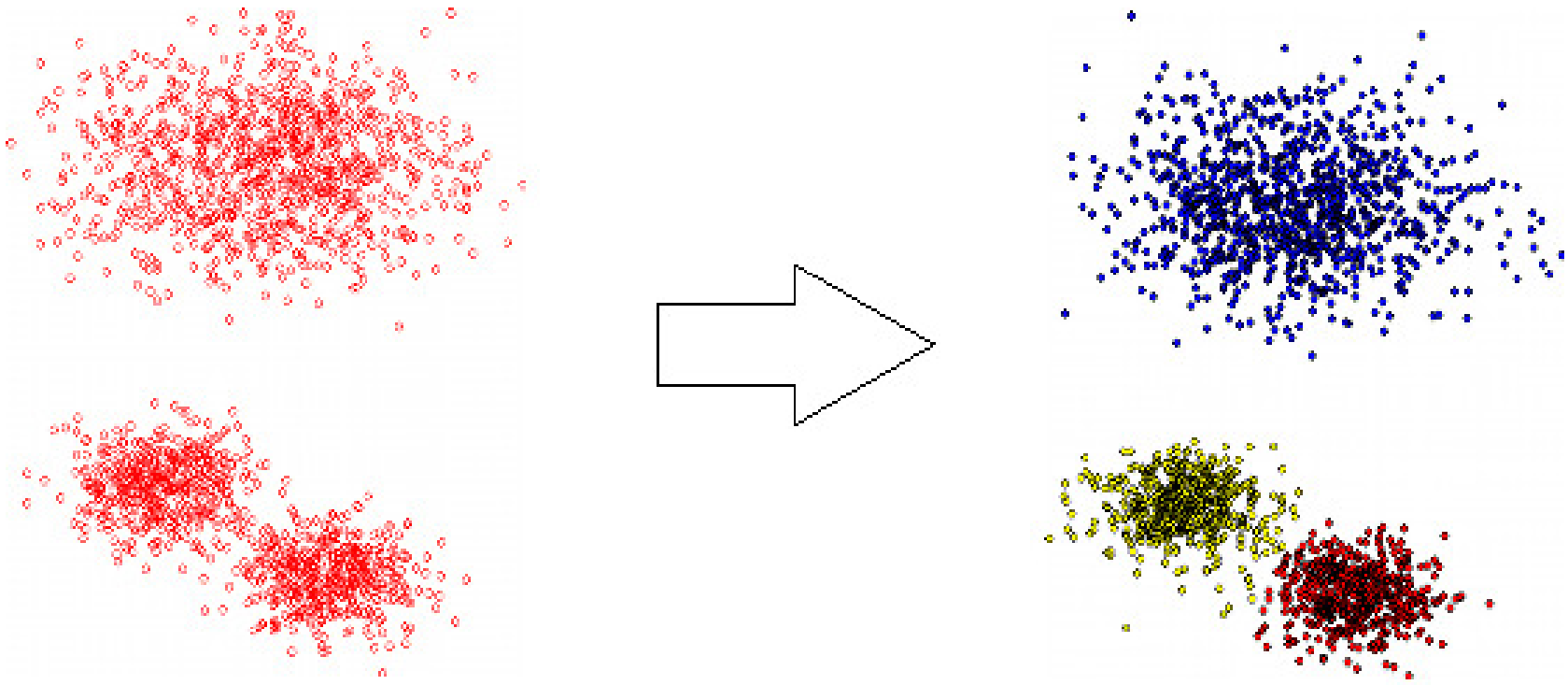
Data partitioning *-or* clustering- *is a Machine Learning task of exploratory data mining the aim of which is to split a data set made of several data (also called objects, data objects, or observations) into several subsets containing homogeneous and similar data. The groups created via this process are called* clusters*.*

Applications of data partitioning

Pattern recognition, web mining, business intelligence, biology, security applications, customer analysis, etc.

Data partitioning : Examples

We want to automatically discover groups of similar data:



Data partitioning : Examples

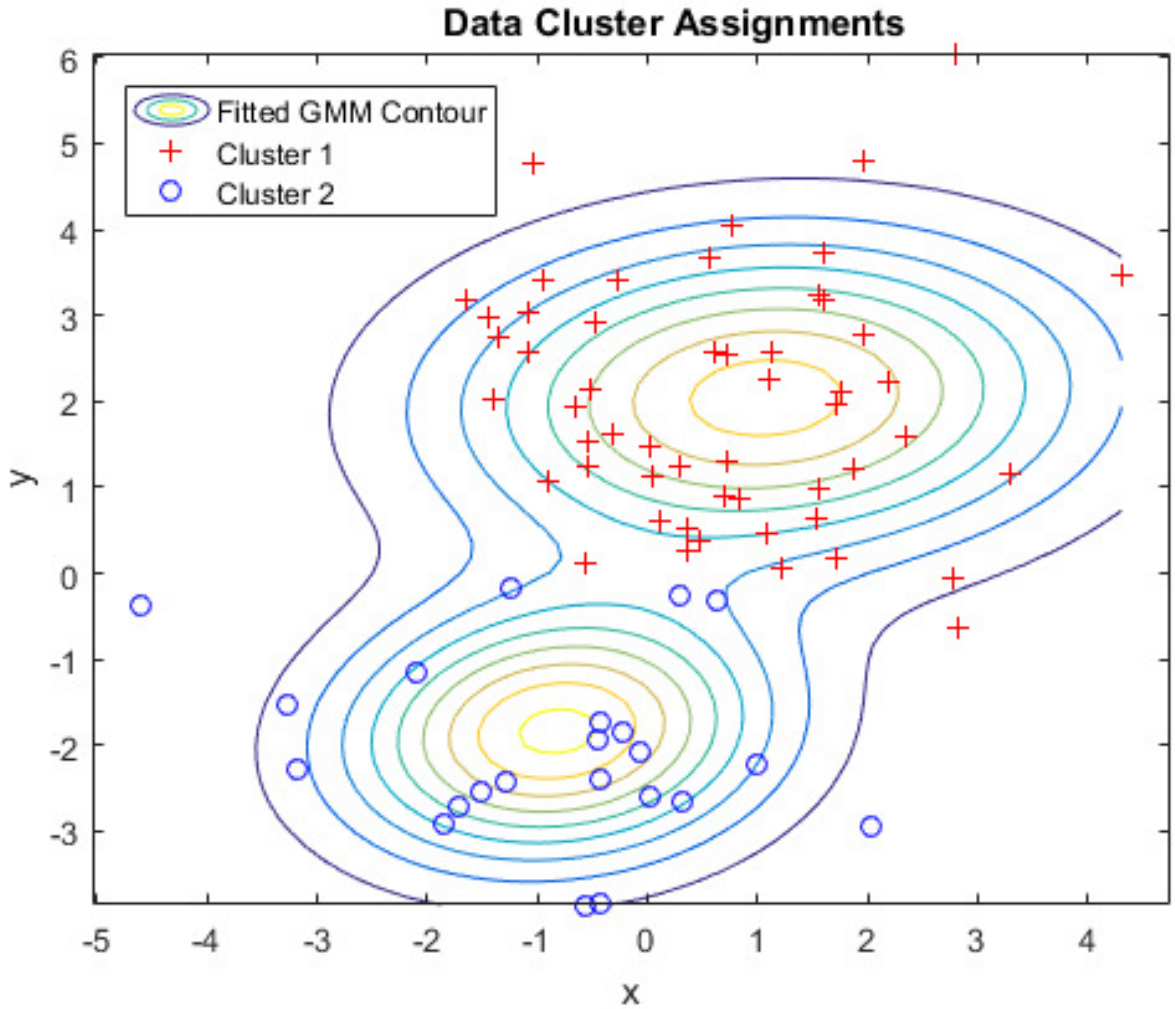


Figure: Example of 2 clusters with Gaussian distributions

Data partitioning : Examples

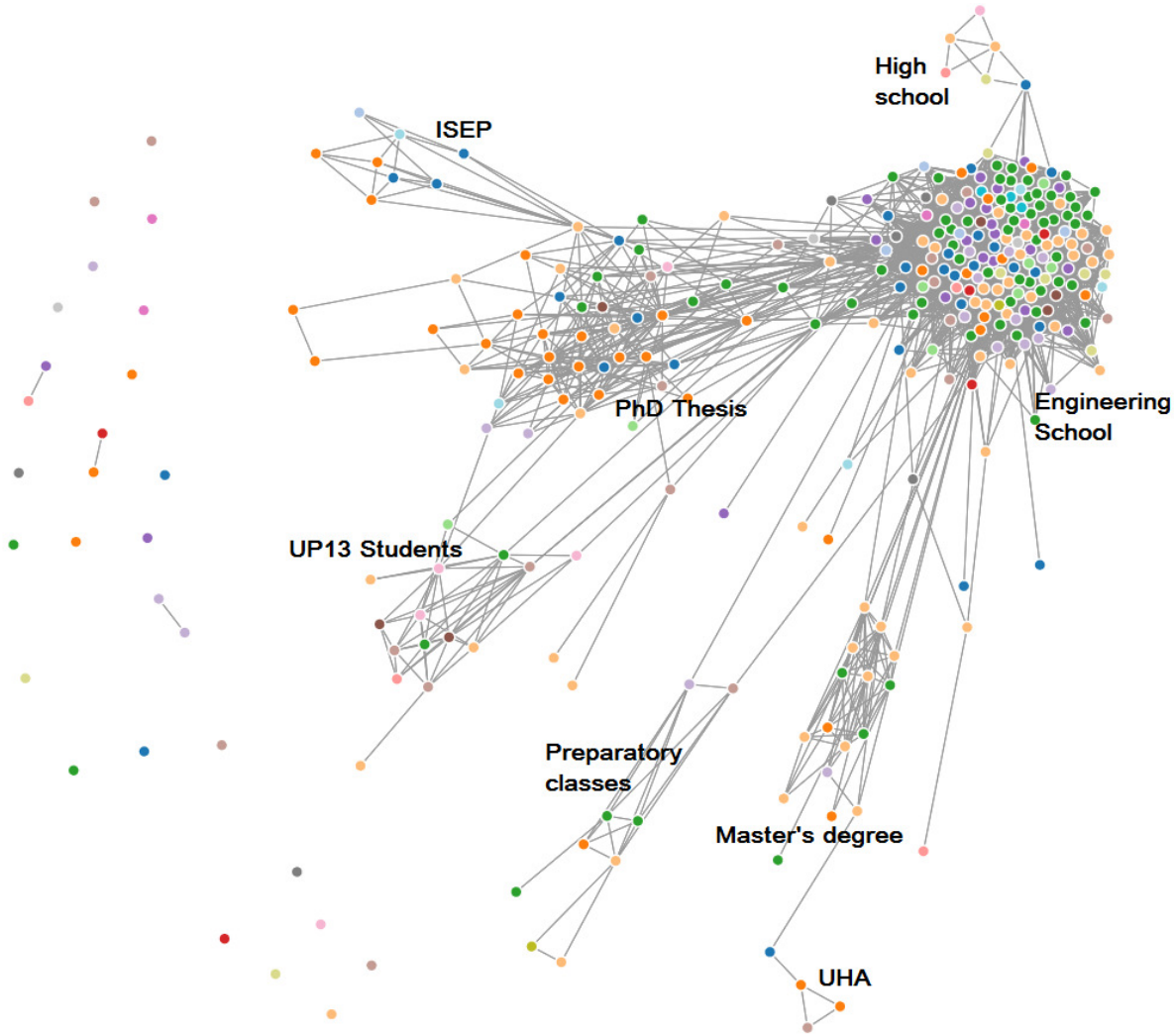


Figure: Community detection in social network graphs

Outline

1 Introduction

2 Clustering

3 Examples of clustering algorithms

4 Open problems in clustering

5 Bibliography

Cluster : Definition

Definition

A cluster is a group of relatively homogeneous data that share more common characteristics between each other than with the data belonging to other clusters.

The notion of cluster therefore relies on the notion of *similarity* and *dissimilarity* between the data.

Clusters and similarity

What is similarity ?

- The quality or state of being similar, the likeness, the resemblance, the a similarity of features.
- How to define the notion of similarity between complex objects described by several attributes ?

Similarity is hard to define, but ...

Clusters and similarity

What is similarity ?

- The quality or state of being similar, the likeness, the resemblance, the a similarity of features.
- How to define the notion of similarity between complex objects described by several attributes ?

Similarity is hard to define, but ...



You know it when you see it !

Clusters, similarity and distance

- The similarity or dissimilarity between two observations is most often computed using a distance function.
- Depending on the data and the application, some distances may prove more appropriate than others (e.g. Hamming distance for text data).

Euclidian distance	$\|\|a-b\|\|_2 = \sqrt{\sum_i (a_i - b_i)^2}$
Squared Euclidian distance	$\|\|a-b\|\|_2^2 = \sum_i (a_i - b_i)^2$
Manhattan distance	$\|\|a-b\|\|_1 = \sum_i \|a_i - b_i\|$
Maximum distance	$\|\|a-b\|\|_\infty = \max_i \|a_i - b_i\|$
Mahalanobis distance	$\sqrt{(a-b)^\top S^{-1}(a-b)}$ where S is the covariance matrix
Hamming distance	$Hamming(a,b) = \sum_i (1 - \delta_{a_i,b_i})$

Table: Examples of common distances

- Creating custom distance functions is sometimes required.

Clustering partitions

Let us denote $X = \{x_1, \cdots, x_N\}, x_n \in \mathbb{R}^d$ a data data set containing N observations described by d real features, and $C = \{c_1, ..., c_K\}$ the set of possible clusters.

Clustering partitions

The result of a clustering algorithm is called a clustering partition*, or just partition.* Depending on whether or not a data can belong to several clusters, the partition can be **hard**, **soft**, or **fuzzy**.

(a) Hard clustering

	c_1	c_2	c_3
x_1	1	0	0
x_2	0	1	0
x_3	0	0	1
x_4	0	0	1

(b) Soft clustering

	c_1	c_2	c_3
x_1	1	1	0
x_2	0	1	1
x_3	0	0	1
x_4	0	0	1

(c) Fuzzy clustering

	c_1	c_2	c_3
x_1	0.9	0.1	0
x_2	0	0.8	0.2
x_3	0	0.3	0.7
x_4	0	0	1.0

Table: Example of several objects' degree of belonging to 3 clusters

Clustering partitions

(a) Hard clustering

	c_1	c_2	c_3
x_1	1	0	0
x_2	0	1	0
x_3	0	0	1
x_4	0	0	1

(b) Soft clustering

	c_1	c_2	c_3
x_1	1	1	0
x_2	0	1	1
x_3	0	0	1
x_4	0	0	1

(c) Fuzzy clustering

	c_1	c_2	c_3
x_1	0.9	0.1	0
x_2	0	0.8	0.2
x_3	0	0.3	0.7
x_4	0	0	1.0

Let us note S the clustering partition:

- In hard clustering, S is a vector $S = \{s_1, \cdots, s_N\}, s_n \in [1..K]$
- In soft clustering, S is a matrix $S = \{s_{n,k}\}_{(N \times K)}, s_{n,k} \in (0, 1)$
- In fuzzy clustering, S is a matrix too:
  $S = \{s_{n,k}\}_{(N \times K)}, s_{n,k} \in [0, 1] \quad \forall n, \sum_k s_{n,k} = 1$

Types of clustering algorithms

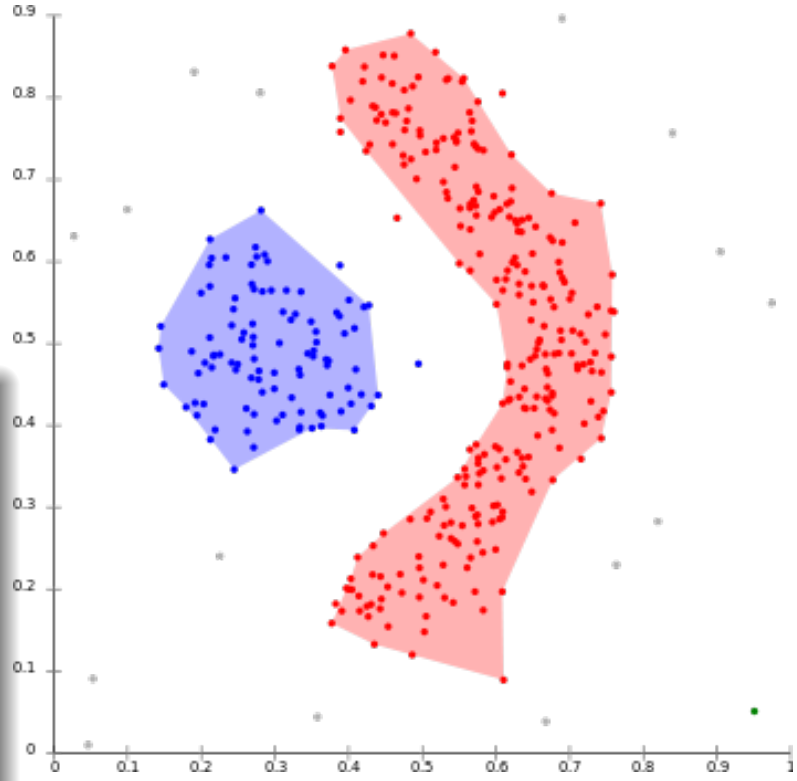
Now that we know what we want to do, the question is: Given a data set, a distance function and a desired type of partition, how do we find the clusters ?

There are several "families" of clustering algorithms that favor different methods to extract the clusters from the data:

- Density-based clustering algorithms
- Hierarchical clustering algorithms
- Prototype-based clustering algorithms
- Distribution-bsed clustering methods
- Spectral clustering methods

Density-based clustering algorithms

- Inspired from the human vision: the clusters are defined as high density spaces and are separated by low density areas.
- Examples of algorithms: DBSCAN, OPTICS

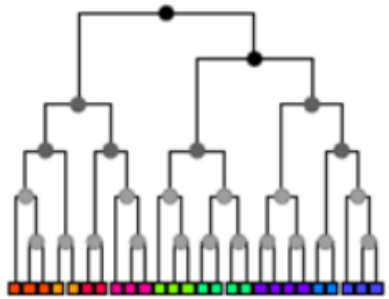


Properties

- Pros: Does not presume the shape of the clusters or their number, detect outliers.
- Cons: High complexity $O(N^2)$, difficult to parametrize

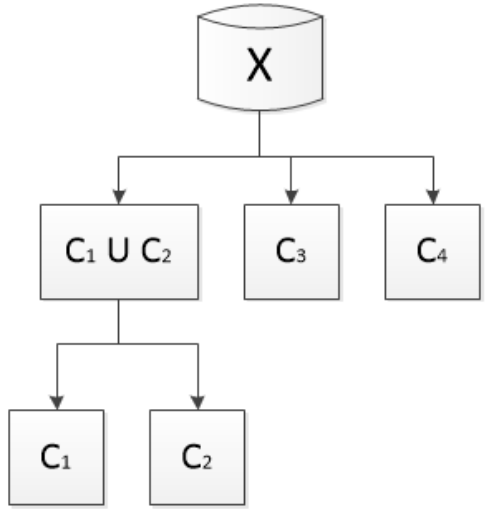
Hierarchical clustering algorithms

- Inspired from phylogenetic trees in biology.
- Approaches: agglomerative, divisive
- Examples of algorithms: HCA, CURE, CLINK



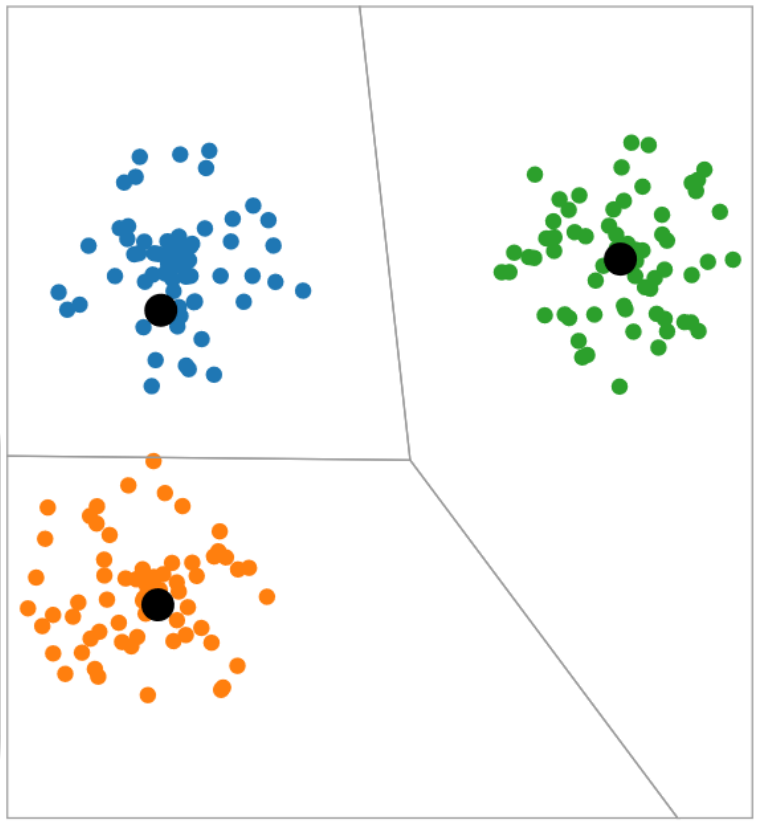
Properties

- Pros: Highlight hierarchical cluster structures, comprehensive visualization of the clusters
- Cons: High complexity $O(N^2 \log N) to O(N^3)$, choosing where to cut the dendrogram can be cumbersome.



Prototype-based clustering algorithms

- Uses vector quantization to sum up the data using representatives (e.g. mean values)
- Examples of algorithms: K-Means, Fuzzy C-Means



Properties

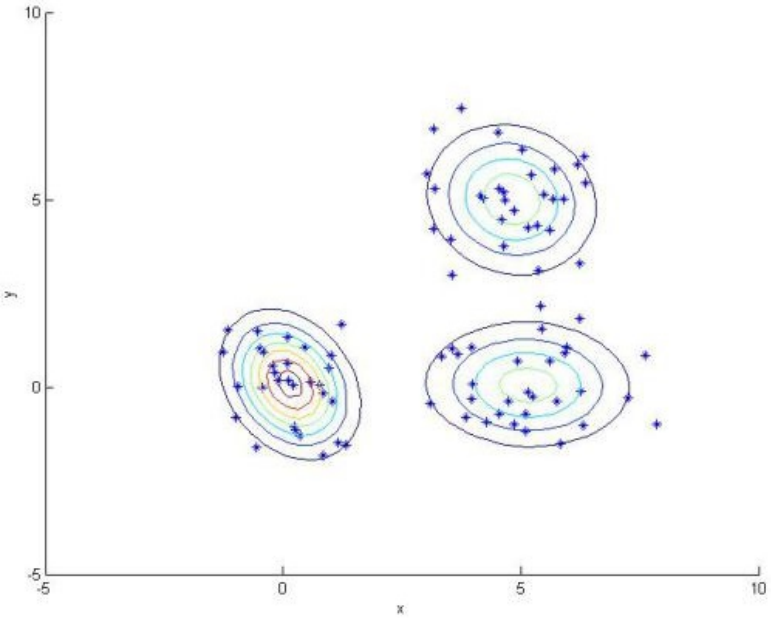
- Pros: Lower complexity $O(N)$
- Cons: Requires to give the number of clusters and to presume of their shapes.

Distribution-based clustering algorithms

- Uses strong mathematical models to describe the clusters (e.g. mixture of gaussian distributions)
- Examples of algorithms: EM

Properties

- Pros: Lower complexity $O(N)$, strong models, convergence proofs
- Cons: Requires to give the number of clusters and to assume that the clusters will follow a given distribution.



Spectral clustering algorithms

- Sees clustering as a graph partitioning problem using the similarity matrix between the different objects.
- Examples of algorithms: Shi-Malik algorithm (normalized cuts), Meila-Shi algorithms.
- Mainly use for image analysis

Properties

- Pros: Good results, does not assume any shape for the clusters, fast with sparse data.
- Cons: High complexity $O(N^3)$, does not scale well, not very intuitive.

Outline

1 Introduction

2 Clustering

3 Examples of clustering algorithms

4 Open problems in clustering

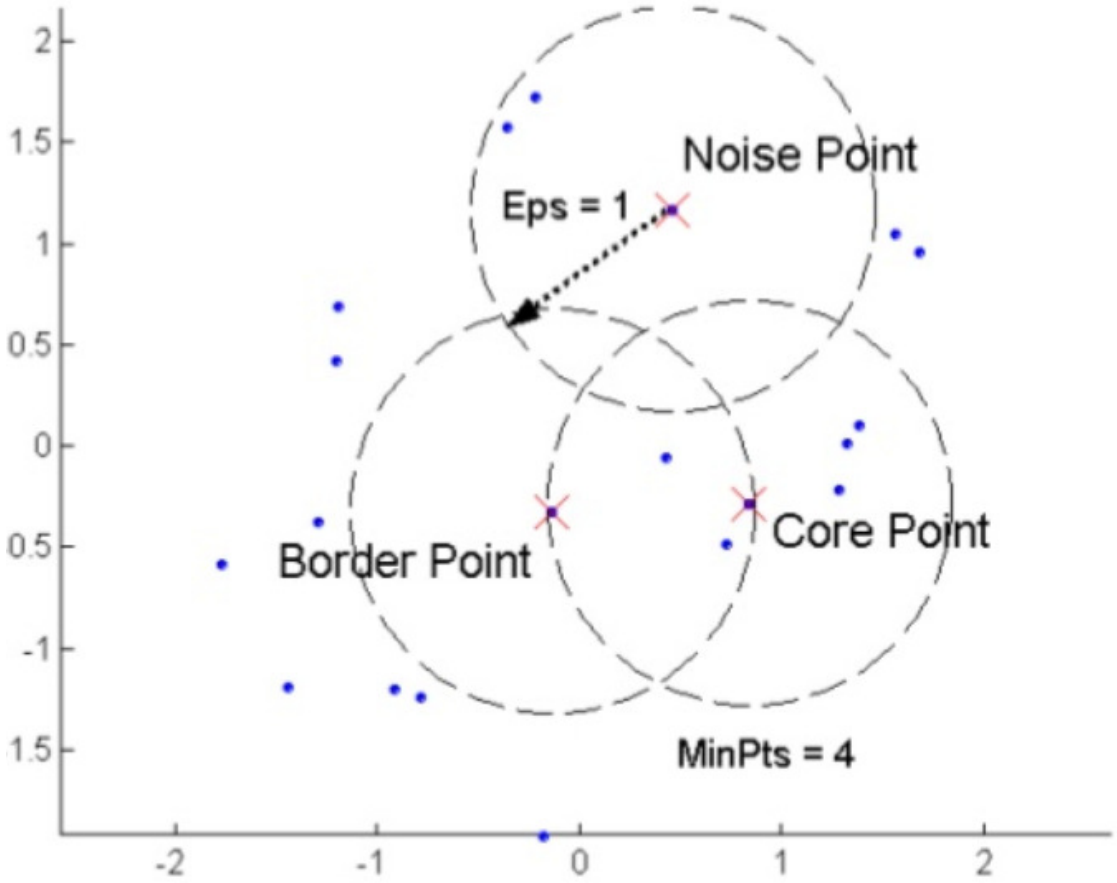
5 Bibliography

DBSCAN: Introduction

DBSCAN is a density based algorithm:

- The "Density" is based on the number of data within the radius "**Eps**" of a given element.
- A point is a **core point** if it has more data in its neighboring radius than a specified number of points "**MinPts**".
  - Core points are the main structures for the clusters.
- A **border point** has fewer points in its radius **Eps** than the number **MinPts** but it is within range of a core point.
- A **noise point** has fewer points in its radius **Eps** than the number **MinPts** and is not within range of a core point.

DBSCAN: Introduction



DBSCAN: Algorithm

Function *DBSCAN*(X,ϵ,*MinPts*)
- C=0
- **forall** $x_n \in X$ **do**
  - **if** x_n has not been visited mark x_n as *visited*
  - V_n= regionQuery(x_n,ϵ)
  - **if** *sizeof*(V_n)$\geq$*MinPts* **then**
    - expandCluster(x_n,V_n,C++,ϵ,MinPts)
  - **else**
    - mark x_n as noise
  - **end**
- **end**

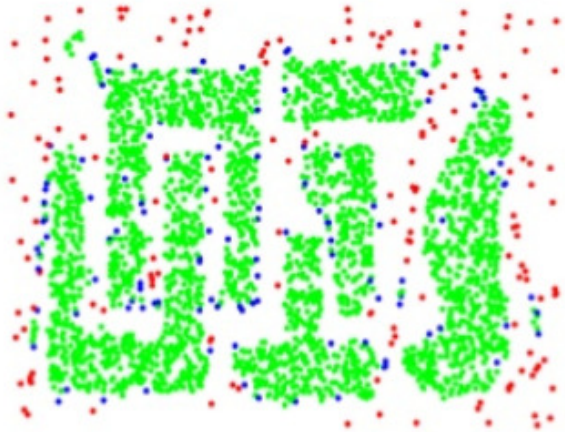
Function *expandCluster*(x_n,V_n,C,ϵ,*MinPts*)
- Add x_n to cluster C
- **forall** $x_i \in V_n$ **do**
  - **if** x_i *has not been visited* **then**
    - Mark x_i as visited
    - V_i = regionQuery(x_i,ϵ)
    - **if** *sizeof*(V_i) $>$ *MinPts* **then**
      - $V_n += V_i$
    - **end**
  - **end**
  - **if**x_i does not belong to any cluster yet: Add x_i to cluster C
- **end**

Function *regionQuery*(x_n,ϵ)
- **forall** $x_i \in X$ **do**
  - **if** $i \neq n$ and $d(x_n, x_i) \leq \epsilon$: list.add($\{x_i\}$)
- **end**
- **return** list

DBSCAN: Example



Original Points



Point types: core, border and noise

Eps = 10, MinPts = 4

DBSCAN: Strengths



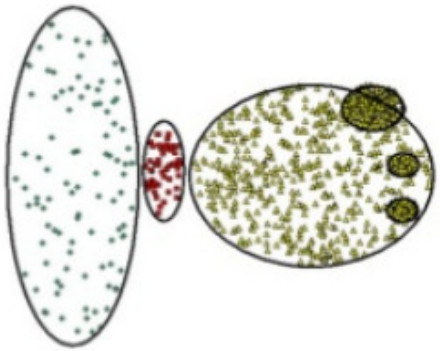
Original Points



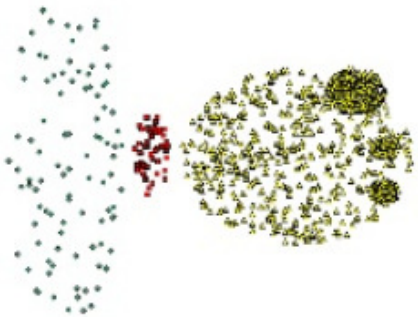
Clusters

- Resistant to noise
- Can handle clusters of different shapes and sizes

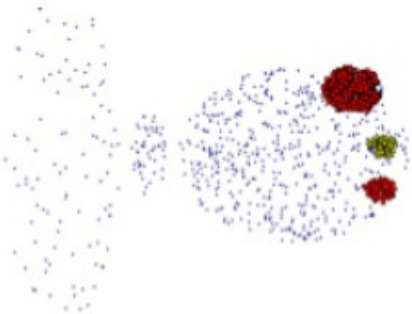
DBSCAN: Limits



Original Points



(MinPts=4, Eps=9.92)



(MinPts=4, Eps=9.75).

- Difficult to parametrize
- High Computational cost
- Does not handle well varying densities
  - OPTICS is a good evolution of DBSCAN that handles varying densities a lot better.

HCA: Introduction

Principle

Create a partition a each step by aggregating together the 2 closest clusters.

- Each data starts out as a single cluster.
- During the process a cluster can be a single data, or a group of data.
- The algorithm return a hierarchy of partitions in the form of a tree containing the history of the different aggregations.

HCA: Algorithm

1. Repeat until there is only one cluster left:
   - 1.a Compute the distance matrix between all existing clusters
   - 1.b Merge the two closest clusters
   - 1.c Update the dendrogram (hierarchical tree)
2. Cut the tree according to a criterion of your choice to get the final partition.

- This algorithm requires distance criterion between clusters: the **linkage criterion**.

HCA: Linkage criteria

Name	Formula	Comments
Single-linkage	$D_s(c_1, c_2) = \min_{x \in c_1, y \in c_2} d(x, y)$	Distance between the two closest elements
Complete-linkage	$D_c(c_1, c_2) = \max_{x \in c_1, y \in c_2} d(x, y)$	Distance between the two farthest element
Average-linkage	$D_a(c_1, c_2) = \frac{1}{\lvert c_1 \rvert \lvert c_2 \rvert} \sum_{x \in c_1} \sum_{y \in c_2} d(x, y)$	Average pairwise distance
Centroid-linkage	$D_\mu(c_1, c_2) = \lvert\lvert \mu_1 - \mu_2 \rvert\rvert$	Distance between the centroids

HCA: Single-Linkage criterion



Original Points



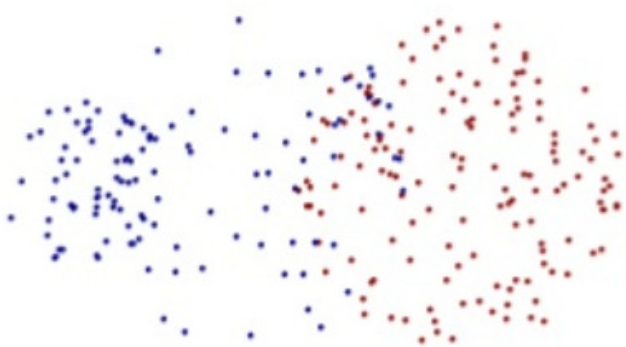
Two Clusters

- Can handle clusters that are not necessarily spherical

HCA: Single-Linkage criterion



Original Points



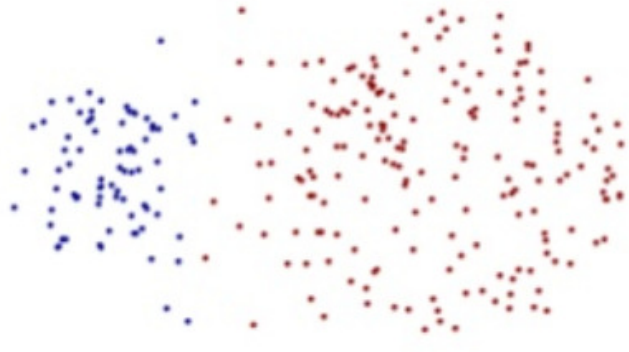
Two Clusters

- Sensitive to noise and outliers
- Cannot handle clusters that are in contact

HCA: Complete-Linkage criterion



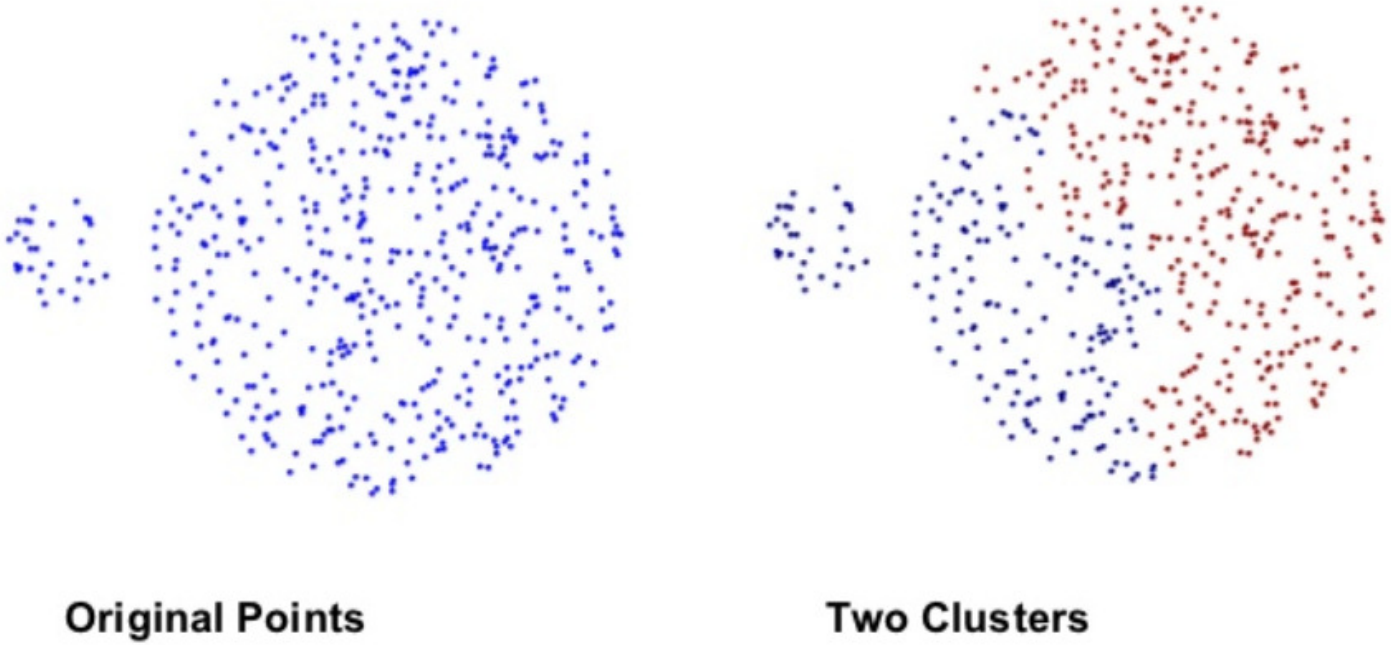
Original Points



Two Clusters

- Less sensitive to noise and outliers

HCA: Complete-Linkage criterion



- Biased toward spherical clusters
- Tends to break large clusters

HCA: Other linkages

Average-linkage

- Less noise sensitive, tends to favor hyper-spherical clusters
- High computational cost for roughly the same results as the centroid-linkage.

HCA: Other linkages

Average-linkage

- Less noise sensitive, tends to favor hyper-spherical clusters
- High computational cost for roughly the same results as the centroid-linkage.

Centroid-linkage

- A good compromise between single and complete link.
- Less sensitive to noise and outliers.
- tends to favor elliptical clusters.

HCA: Other linkages

Average-linkage

- Less noise sensitive, tends to favor hyper-spherical clusters
- High computational cost for roughly the same results as the centroid-linkage.

Centroid-linkage

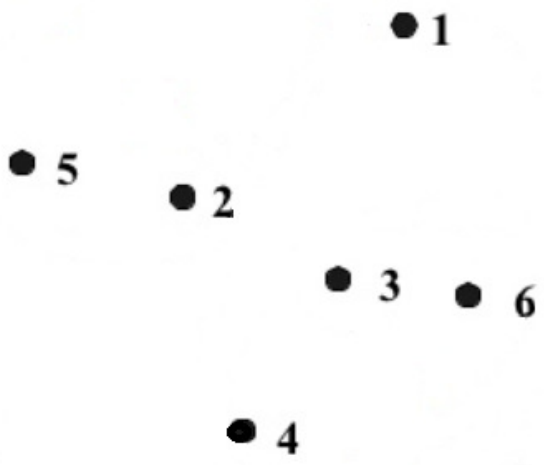
- A good compromise between single and complete link.
- Less sensitive to noise and outliers.
- tends to favor elliptical clusters.

Ward's method

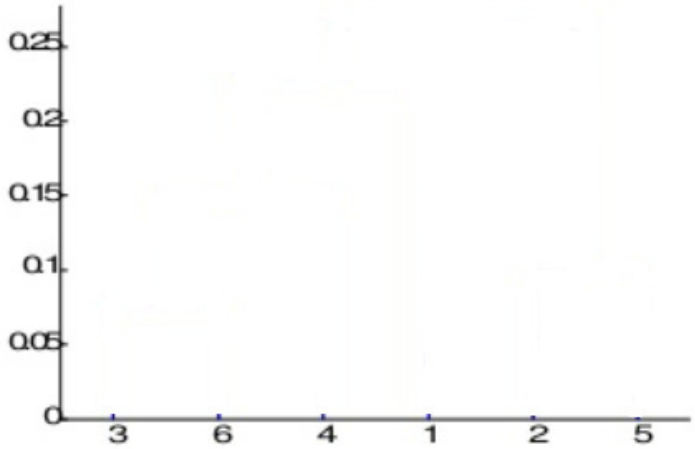
This similarity is based on the increase on the mean squared distance to the centroid when 2 clusters are merged.

- Less sensitive to noise and outliers.
- tends to favor elliptical clusters.

HCA: Example

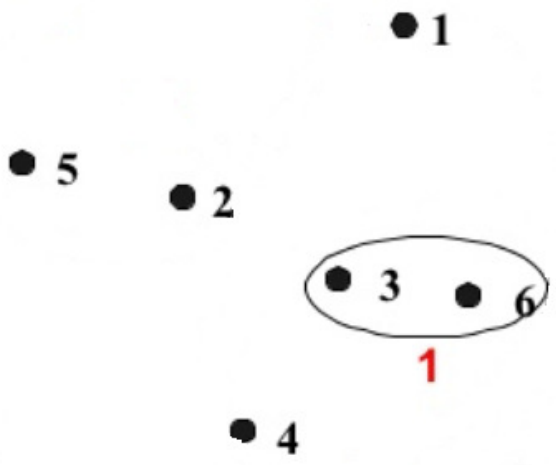


Nested Clusters

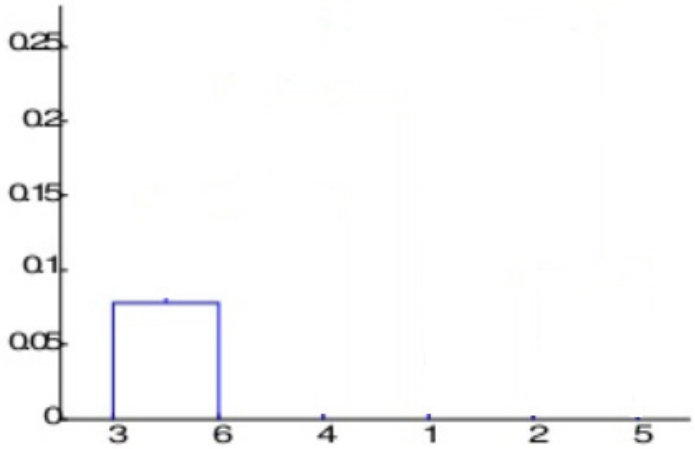


Dendrogram

HCA: Example

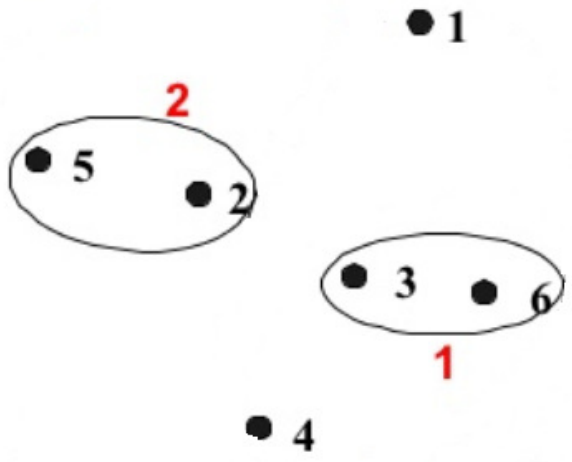


Nested Clusters

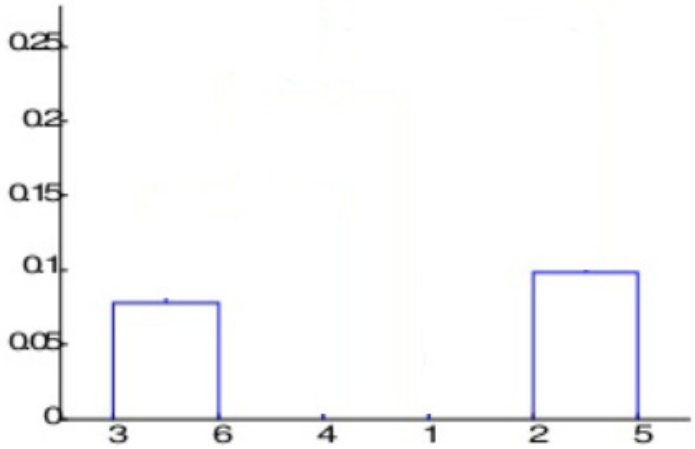


Dendrogram

HCA: Example

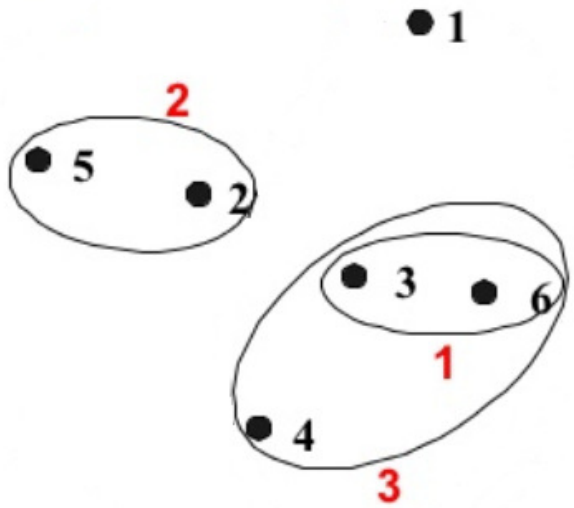


Nested Clusters

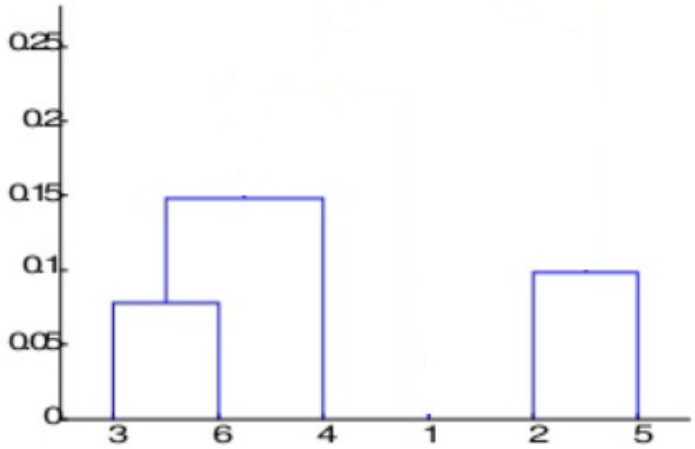


Dendrogram

HCA: Example

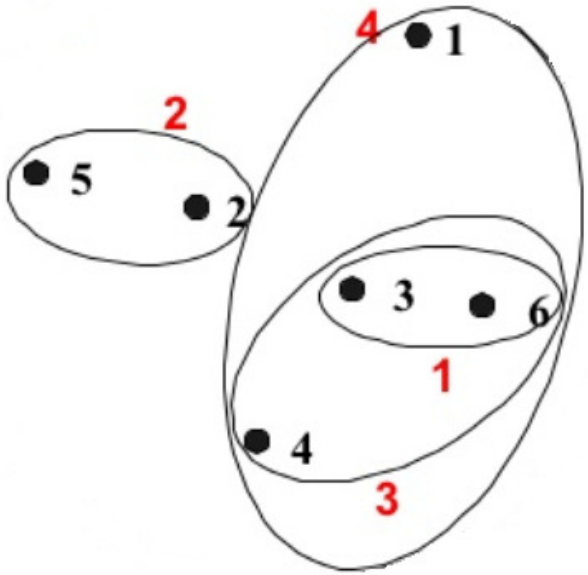


Nested Clusters

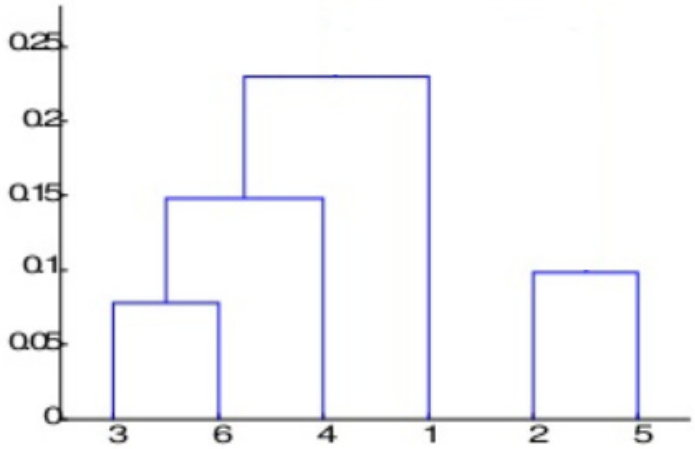


Dendrogram

HCA: Example

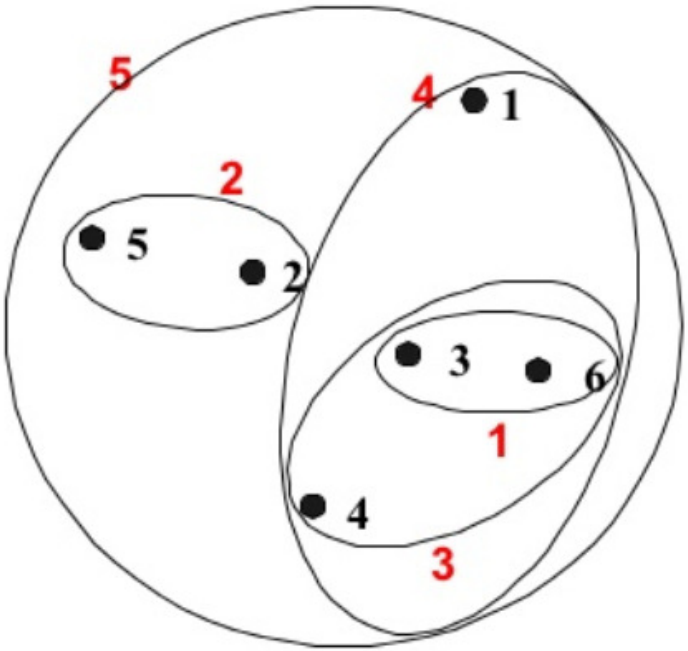


Nested Clusters

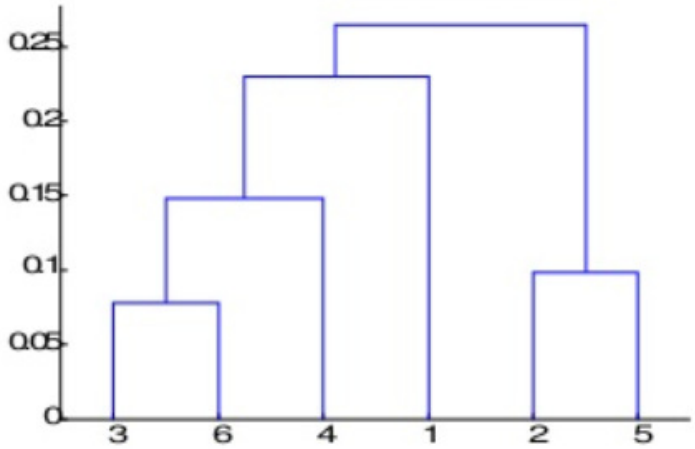


Dendrogram

HCA: Example

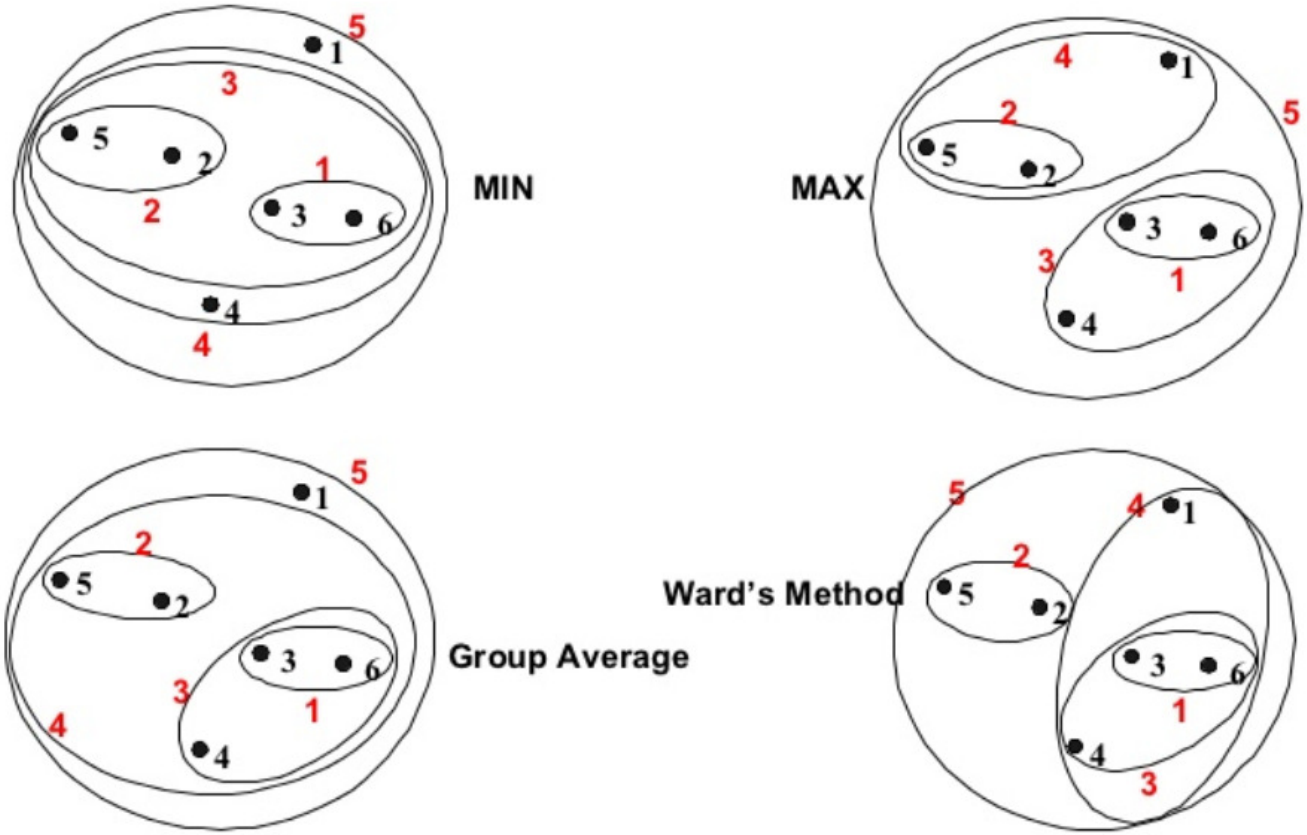


Nested Clusters



Dendrogram

HCA: Examples comparisons



K-Means: Introduction

The method of the K Means, or K-Means algorithm, is a special case of the mobile centers methods. Its principle consists in using a certain number of representatives (centroids, or prototypes) in the data space. Each of these representative will represent a cluster.

- In the end, each data is associated to its closest prototype in order to obtain the clustering partition.
- The groups formed with this process are homogeneous and well separated.

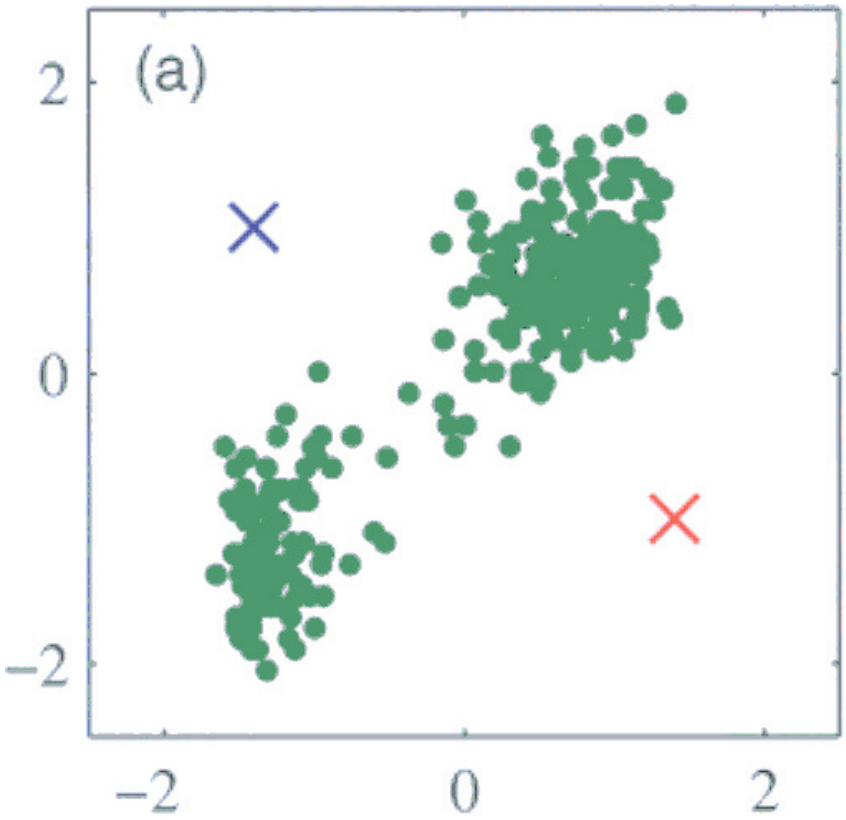
K-Means: Algorithm

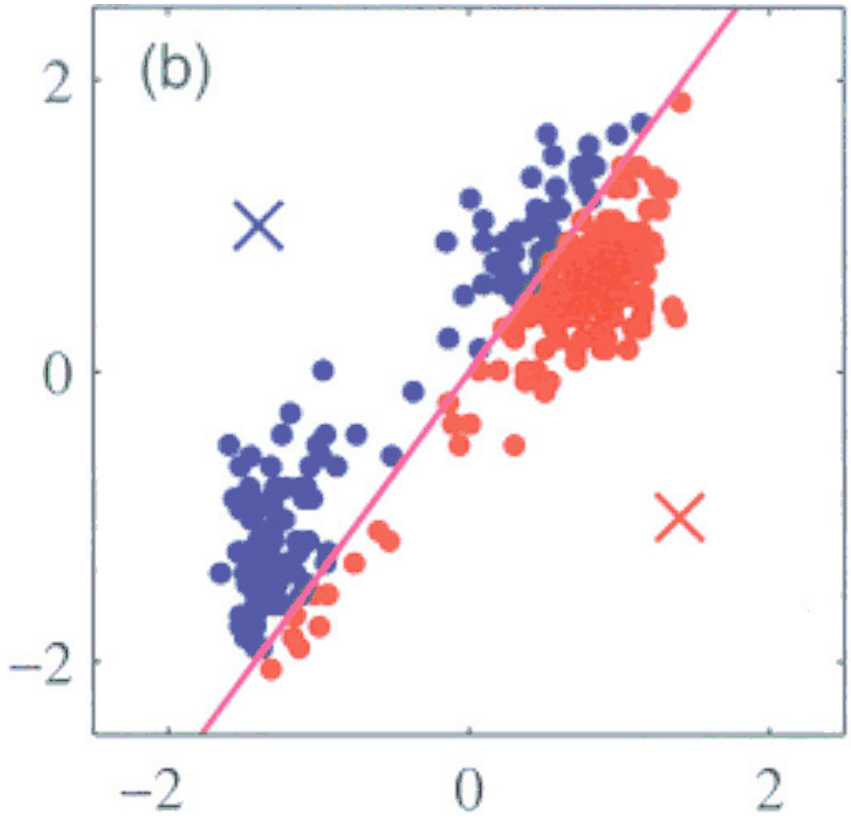
1. Randomly select K initial center.
2. Assign each data x to its closest center μ_i.
3. If the partition doesn't change: **stop**
4. Else, update the centers, each μ_i must be the gravity center of its cluster: $\mu_i = \frac{1}{|c_i|} \sum_{x \in c_i} x$
5. Go to 2

Like in most clustering algorithms, step 2 is dependent on a chosen distance function. For the K-Means algorithm, the Euclidian distance is usually prefered:

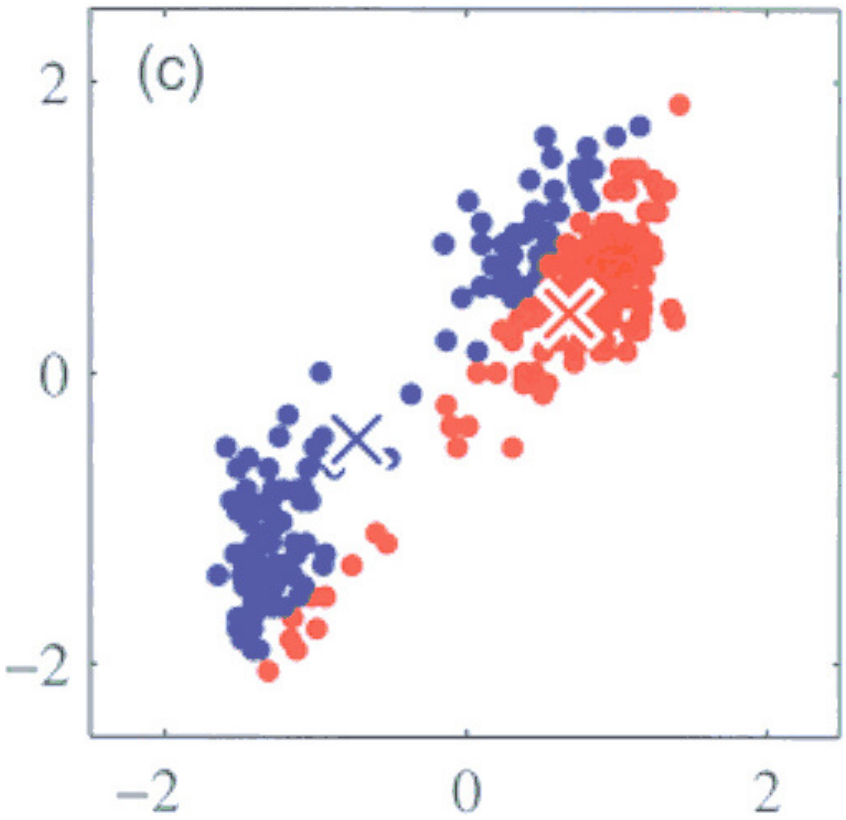
$$d(x, y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

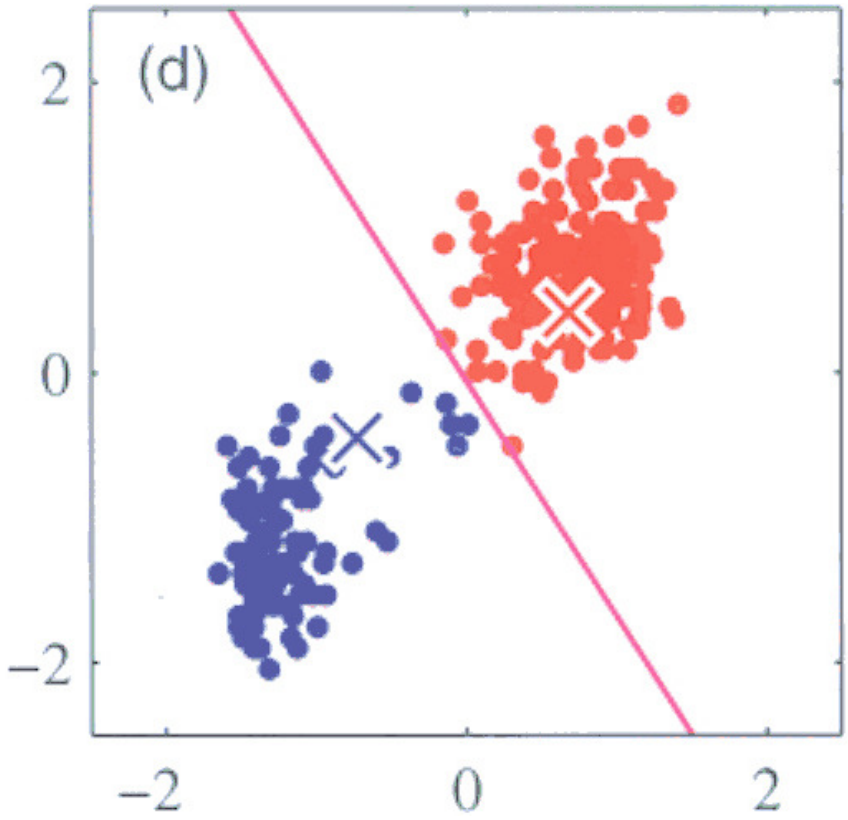
K-Means: Example



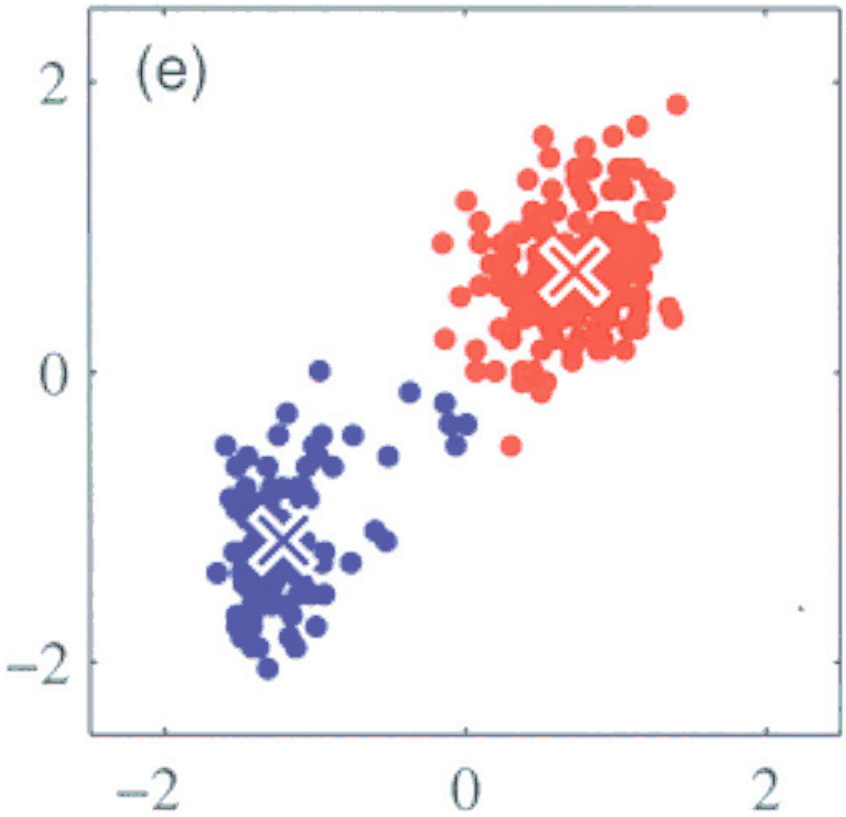


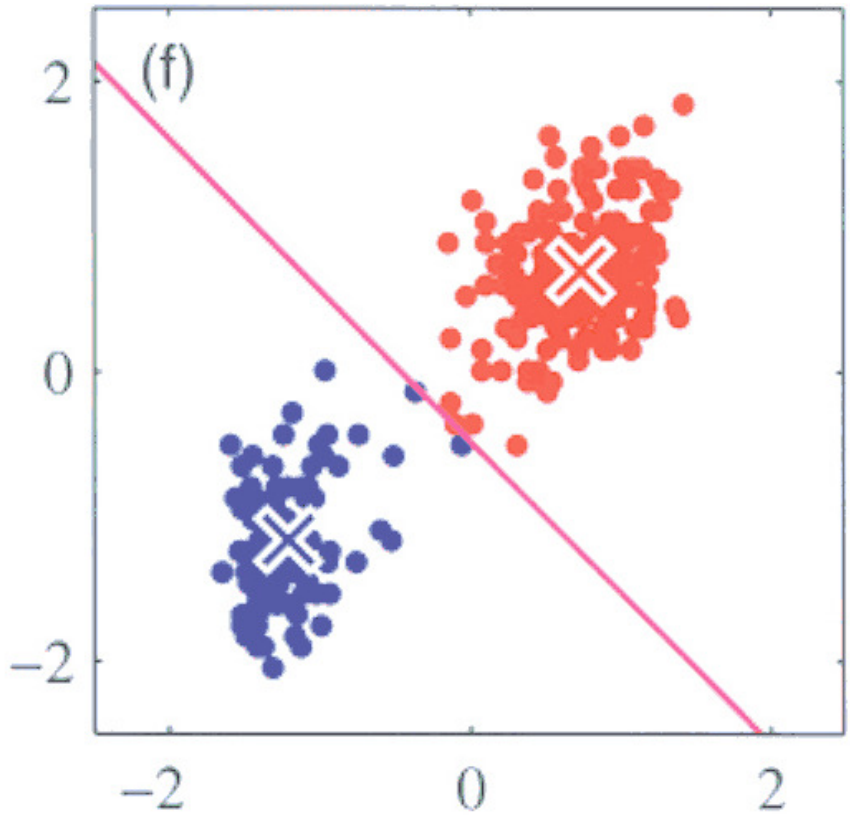
K-Means: Example



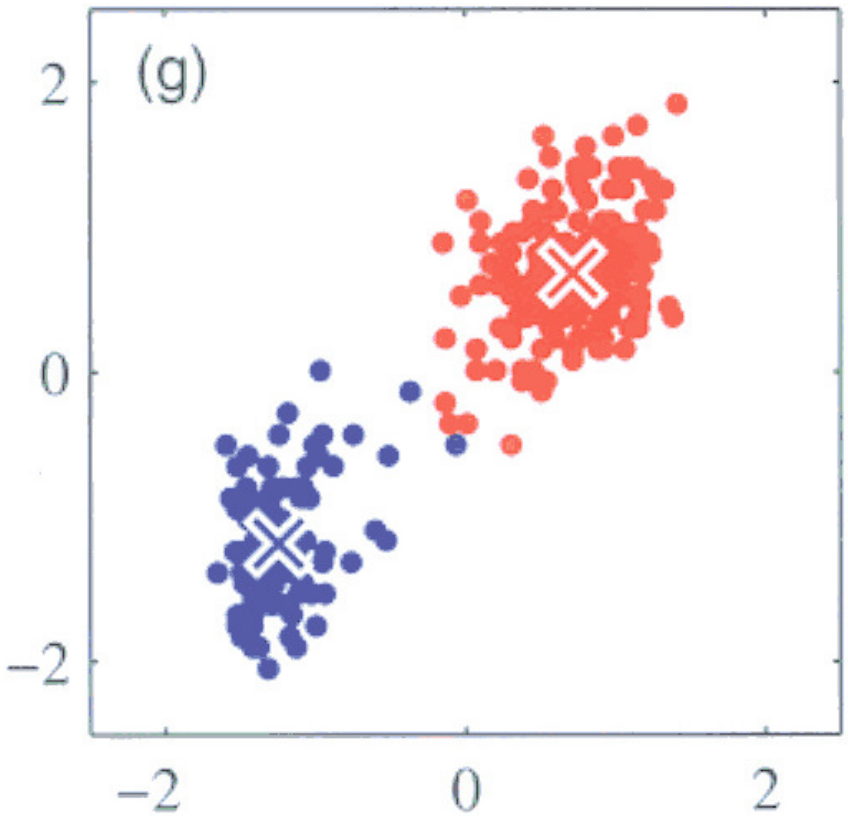


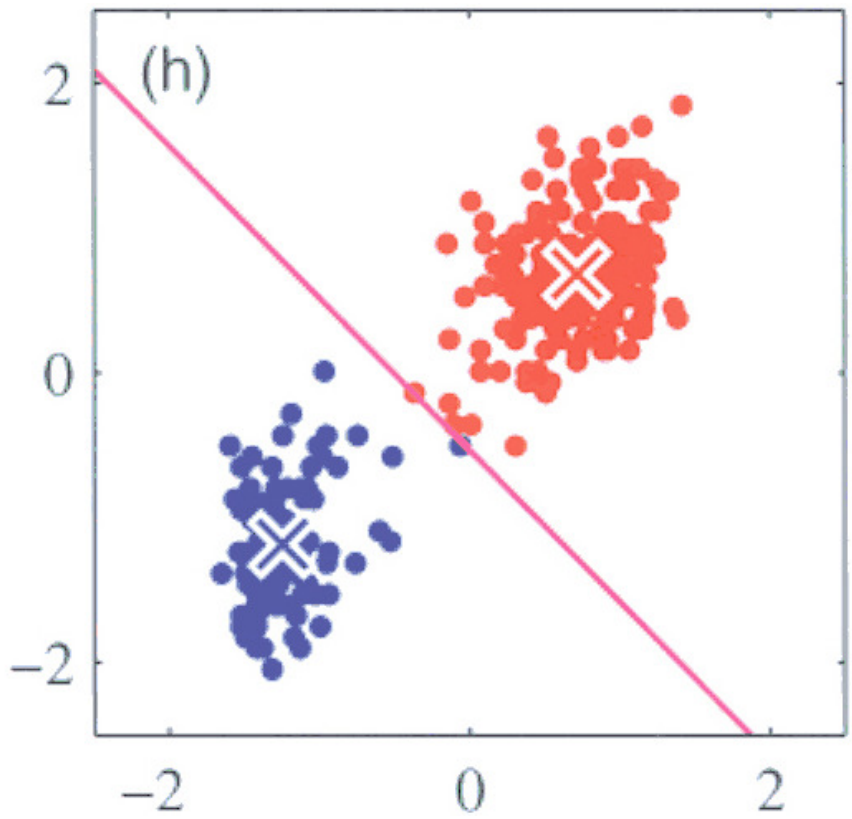
K-Means: Example





K-Means: Example





K-Means: Properties

- Guaranteed to monotonically decrease average squared distance in each iteration

$$L(\mu)^{(t)} = \sum_{i=1}^{N} \min_j ||\mu_j - x_i||_2^2$$

$$L(\mu)^{(t+1)} \leq L(\mu)^{(t)}$$

K-Means: Properties

- Guaranteed to monotonically decrease average squared distance in each iteration

$$L(\mu)^{(t)} = \sum_{i=1}^{N} \min_j ||\mu_j - x_i||_2^2$$

$$L(\mu)^{(t+1)} \leq L(\mu)^{(t)}$$

- Convergence to a local minimum

K-Means: Properties

- Guaranteed to monotonically decrease average squared distance in each iteration

$$L(\mu)^{(t)} = \sum_{i=1}^{N} \min_j ||\mu_j - x_i||_2^2$$

$$L(\mu)^{(t+1)} \leq L(\mu)^{(t)}$$

- Convergence to a local minimum
- Algorithmic complexity at each iteration : $O(n \cdot d \cdot K)$

K-Means: Properties

- Guaranteed to monotonically decrease average squared distance in each iteration

$$L(\mu)^{(t)} = \sum_{i=1}^{N} \min_j ||\mu_j - x_i||_2^2$$

$$L(\mu)^{(t+1)} \leq L(\mu)^{(t)}$$

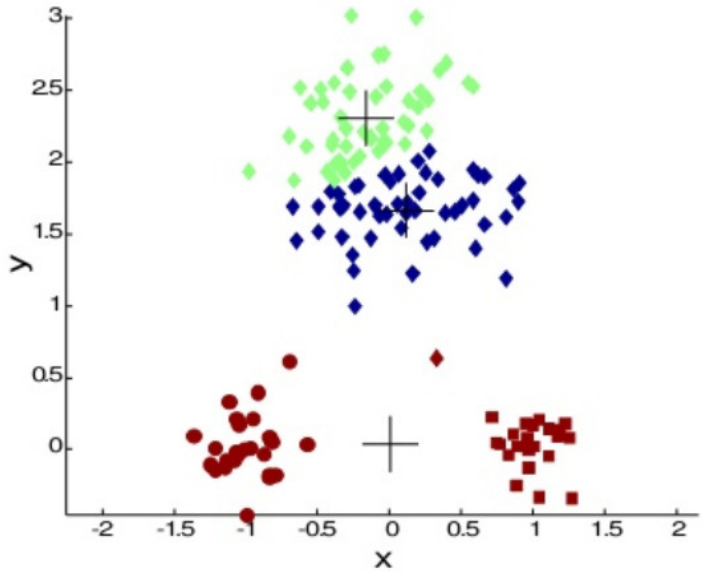
- Convergence to a local minimum
- Algorithmic complexity at each iteration : $O(n \cdot d \cdot K)$
- Non-convex optimization : **NP-hard problem**

K-Means: Limits

Instability

The K-Means algorithm is highly dependent on the initialization of the centers.

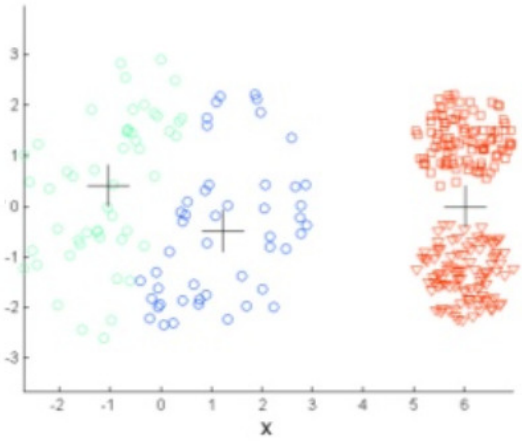
- This problem can be reduced by running the algorithm several time.
- It is also possible to initialize the centers using another clustering algorithm.



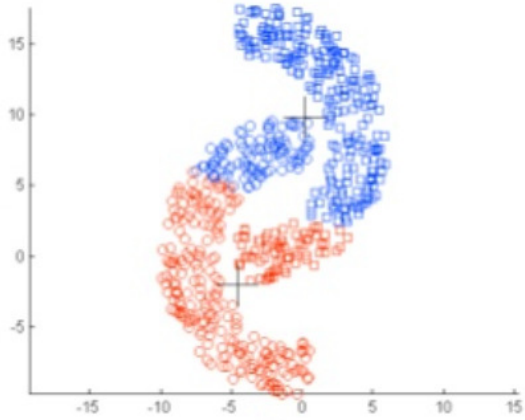
K-Means: Limits

Clusters shapes, size and density

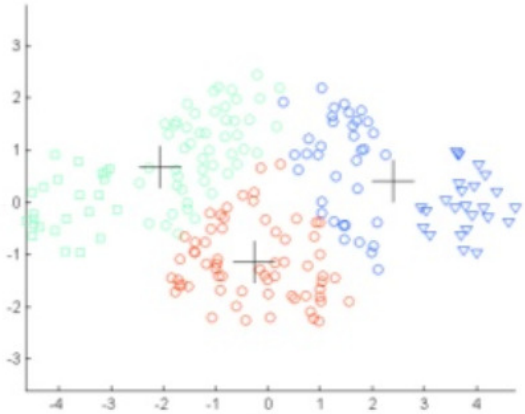
- The K-Means algorithm can only capture spherical or almost spherical clusters. Any non-convex shape will be missed.
- Clusters with different density or sizes can also be difficult to find using the K-Means algorithm.



K-means (3 Clusters)



K-means (2 Clusters)



K-means (3 Clusters)

K-Means: Limits

Choosing the number of clusters

The K-means algorithms requires the number of clusters "K" to be provided as a parameter. In purely exploratory data mining tasks, this number is not always known and must be guessed.

- Sometimes the only solution is to try several values for K and to see which gives the best results.